

Clase 6:

Estructuras de Datos : Arreglos

Hasta ahora estuvimos trabajando con estructuras de control como if, switch, while y for. Con ellas aprendimos a tomar decisiones, repetir acciones y controlar cómo se ejecuta un programa. En otras palabras, aprendimos a decirle al programa **qué hacer y cuándo hacerlo**.

Pero hay una pregunta importante que todavía no respondimos del todo:

¿Qué pasa cuando necesitamos trabajar con muchos datos al mismo tiempo?

Por ejemplo:

- Guardar las notas de 30 alumnos.
- Registrar 50 ventas.
- Almacenar los precios de 100 productos.

¿Vamos a crear 30, 50 o 100 variables distintas?

Claramente, eso no sería práctico.

Aquí es donde entran en juego las **estructuras de datos**. Estas nos permiten organizar la información de una manera más ordenada y eficiente. No se trata solo de guardar datos, sino de guardarlos de forma que podamos trabajar con ellos fácilmente.

En este apunte vamos a empezar a explorar este nuevo concepto. Veremos qué son las estructuras de datos y por qué son tan importantes en programación. Luego nos enfocaremos en la más simple y fundamental de todas: **el arreglo**.

A partir de ahora vamos a combinar lo que ya sabemos sobre estructuras de control con nuevas formas de almacenar información. Este es un paso muy importante, porque a medida que los programas crecen, la forma en que organizamos los datos se vuelve tan importante como las instrucciones que escribimos.

¿Qué es una estructura de datos?

Una **estructura de datos** es una forma de organizar, gestionar y almacenar información en una computadora, de manera que sea posible acceder a los datos y modificarlos de forma eficiente.

En otras palabras: una estructura de datos define **cómo se guardan los datos** y **qué operaciones pueden hacerse sobre ellos** (por ejemplo, agregar, eliminar, buscar u ordenar).

El objetivo principal de una estructura de datos no es únicamente almacenar información, sino hacerlo siguiendo una organización determinada que facilite operaciones como:

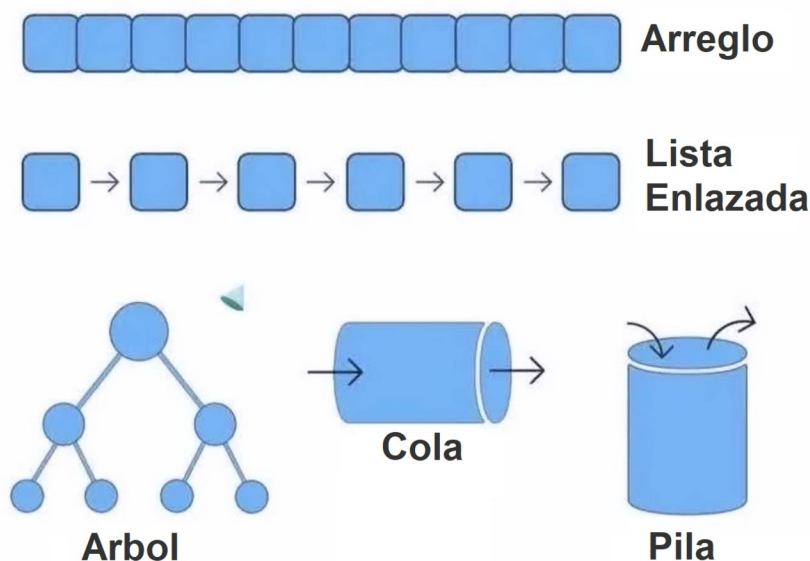
- Recorrer los datos.
- Buscar un elemento específico.
- Modificar valores.
- Realizar calculos.
- Insertar o eliminar información.

La manera en que los datos se organizan influye directamente en la complejidad y eficiencia del programa. Una estructura bien elegida permite resolver un problema de forma más clara y ordenada.

Existen distintos tipos de estructuras de datos, cada una diseñada para resolver necesidades específicas. Algunas de las más conocidas son:

- Arreglos (arrays)
- Listas enlazadas (linked lists)
- Pilas (stack)
- Colas (queues)
- Árboles (trees)
- Tablas hash (hash table)

En esta primera aproximación, nos centraremos en la estructura más simple y fundamental: el **arreglo**, que servirá como base para comprender estructuras más complejas en el futuro.



¿Que es un arreglo?

Un **arreglo** (array) es una estructura de datos que permite almacenar un conjunto de elementos del mismo tipo en posiciones contiguas de memoria.

Cada elemento del arreglo se identifica mediante un índice numérico, que indica su posición dentro de la estructura.

Un arreglo permite trabajar con múltiples datos relacionados utilizando un único identificador.

Características principales

Un arreglo posee las siguientes características fundamentales:

1. **Homogeneidad**

Todos los elementos del arreglo deben ser del mismo tipo de dato (int, float, char, etc.).

2. **Tamaño fijo**

El tamaño del arreglo se define en el momento de su declaración y no puede modificarse durante la ejecución del programa.

3. **Memoria contigua**

Los elementos se almacenan en posiciones consecutivas de memoria.

4. **Acceso mediante índice**

Cada elemento se accede utilizando un índice numérico.

5. **Indexación y relación con el tamaño**

En los arreglos, la indexación comienza en 0. Esto significa que el primer elemento se encuentra en la posición 0 y no en la posición 1.

Si un arreglo tiene tamaño **n**, entonces:

- El primer índice válido es **0**.
- El último índice válido es **n - 1**.

Ejemplo: Un arreglo de 5 números.

	Numeros[5]				
Valor:	10	20	30	40	50
Indice:	0	1	2	3	4

En este caso:

Tamaño = 5

Último índice = $5 - 1 = 4$

Por lo tanto, los indices validos serian:

Numeros[0], Numeros[1], Numeros[2], Numeros[3], Numeros[4].

Arreglos en C++

Declaración de un arreglo

En C++, la sintaxis básica para declarar (crear) un arreglo es:

```
Tipo Nombre[Tamaño];
```

Por ejemplo:

```
int edades[10];  
float promedios[5];  
char letras[26];
```

Importante:

- El tamaño indica la **cantidad de elementos** que puede guardar.
- Los índices van desde 0 hasta tamaño – 1.
- El **tamaño** debe ser un número entero o una constante (no puede ser una variable en arreglos estáticos)

Inicialización de un arreglo

Se pueden asignar valores iniciales en el momento de la declaración:

```
int Edades[5]={18, 19, 20, 21, 22};
```

Si queremos ver el contenido de cada elemento individualmente, podemos hacerlo de la siguiente forma:

```
cout<<Edades[0]; // 18
```

```
cout<<Edades[1]; // 19
```

```
cout<<Edades[2]; // 20
```

```
cout<<Edades[3]; // 21
```

```
cout<<Edades[4]; // 22
```

Si se inicializa con menos valores, el resto se completa con 0 (para tipos numéricos):

```
int Edades[5]={18, 19}; // Resultado: {18, 19, 0, 0, 0}
```

Si no se especifica el tamaño, C++ lo deduce automáticamente:

```
int Edades[]={18, 19, 20}; // Crea un arreglo de 3 elementos
```

Si no se le asigna un valor a los distintos elementos del arreglo, estos guardaran datos basura que se encuentran en la memoria.

```
int Edades[5]; // Edades[0]=Basura
```

Es necesario inicializar un arreglo en 0 (o en algún valor definido) porque, si no se hace, sus elementos contienen **valores indeterminados**. Hay dos formas de inicializar en 0:

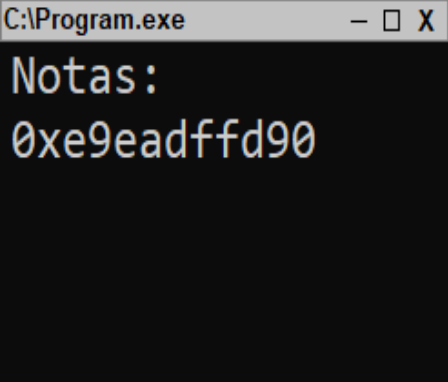
```
int Edades[]={0} O int Edades[0];
```

Como recorrer un arreglo

Un error muy común al empezar es pensar que podemos mostrar todo el contenido de un arreglo con una sola línea como `cout << notas;`. Pero esto no funciona como se espera.

Por ejemplo:

```
int main() {  
    // Arreglo de 5 elementos  
    int notas[5] = {7, 9, 6, 8, 10};  
  
    cout<<"Notas:"<<endl;  
    cout<<notas;
```



Lo que vemos en pantalla no son los valores del arreglo, sino la **dirección de memoria** donde comienza a guardarse el arreglo. Esto se debe a que, en C++, el nombre del arreglo actúa como un puntero a su primer elemento, y `cout` lo interpreta como una dirección, no como una lista de valores.

Entonces, ¿cómo hacemos para ver cada elemento del arreglo? La respuesta es simple: no hay una forma automática de mostrar todo el arreglo de una sola vez. Debemos **recorrerlo posición por posición**, accediendo a cada elemento mediante su índice, y mostrar cada valor individualmente.

Para eso utilizamos un bucle. El bucle `for` es la herramienta ideal porque nos permite usar una variable como índice que vaya avanzando desde el primer elemento hasta el último. A continuación vemos cómo se hace exactamente:

Recorrer un arreglo con un bucle for

Para recorrer un arreglo completo se utiliza un bucle `for` que:

1. Comienza en el índice 0 (primer elemento).
2. Continúa mientras el índice sea **menor que el tamaño** del arreglo.
3. Avanza al siguiente índice en cada iteración (`i++`).

De esta forma, la variable del bucle (`i`) toma los valores 0, 1, 2, ..., hasta tamaño-1, que son exactamente todos los índices válidos del arreglo.

```
int main() {  
    // Arreglo de 5 elementos  
    int notas[5] = {7, 9, 6, 8, 10};  
  
    // Recorrido completo con un for  
    cout<<"Notas:"<<endl;  
    for(int i = 0; i < 5; i++) {  
        cout << notas[i] << endl;  
    }  
  
    return 0;  
}
```

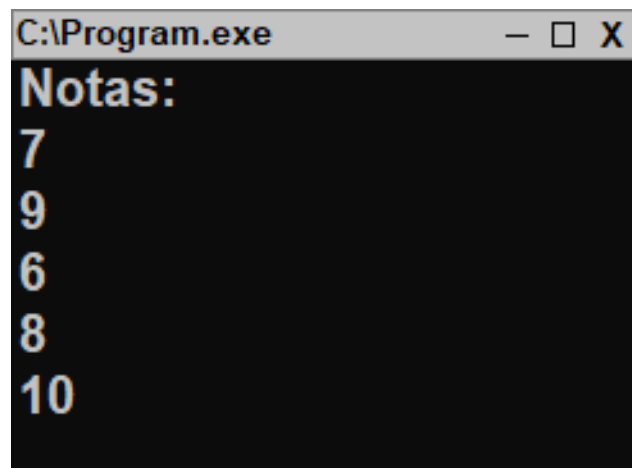
Primero, dentro de la función *main*, se declara un arreglo llamado *notas* que guarda 5 números enteros: 7, 9, 6, 8 y 10.

Antes de recorrerlo, aparece la línea `cout<<"Notas:"<<endl;` que simplemente muestra en pantalla la palabra "Notas:" y luego baja a la siguiente línea.

Luego viene el bucle for. Allí se declara una variable *i* que empieza valiendo 0. El bucle pregunta: "¿i es menor que 5?". Como 0 es menor que 5, entra al bloque y ejecuta `cout<<Notas[i]<<endl;`. En este momento *i* vale 0, así que muestra el contenido de *notas*[0], que es 7, y luego baja una línea.

Terminada esa vuelta, *i* aumenta a 1. El bucle vuelve a preguntar "¿1 es menor que 5?" y al ser verdadero, muestra *notas*[1] que es 9. Así continúa: cuando *i* vale 2 muestra 6, cuando vale 3 muestra 8, y cuando vale 4 muestra 10.

Finalmente *i* aumenta a 5. El bucle pregunta "¿5 es menor que 5?" y la respuesta es falsa, entonces sale del bucle. Luego `return 0;` termina el programa.



```
C:\Program.exe
Notas:
7
9
6
8
10
```

Operaciones con arreglos

Como se observa en el código anterior, el ciclo for recorre el arreglo accediendo a cada uno de sus elementos mediante el índice. En cada iteración, la variable *i* representa una posición válida del arreglo, permitiéndonos obtener el valor almacenado en *notas[i]*.

Aprovechando esta característica, podemos realizar operaciones sobre todos los elementos del arreglo, como por ejemplo calcular un promedio.

Si quisiéramos calcular el promedio de notas de un curso de 20 alumnos, el procedimiento sería el mismo: recorrer el arreglo completo, acumular todas las notas y luego dividir por la cantidad total de alumnos.

El algoritmo sería el siguiente:

```
int main() {
    int notas[20];
    int suma = 0;

    // Carga de datos
    for(int i = 0; i < 20; i++) {
        cin >> notas[i];
    }

    // Acumulación
    for(int i = 0; i < 20; i++) {
        suma += notas[i];
    }

    // Cálculo del promedio
    double promedio = (double)suma / 20;

    cout << "Promedio del curso: " << promedio << endl;

    return 0;
}
```

El cálculo del promedio se basa en la fórmula matemática:

Promedio = suma de los valores / cantidad de valores

En este caso:

- La suma se obtiene recorriendo el arreglo y acumulando cada nota.
- La cantidad de valores es el tamaño del arreglo (20).

Es importante notar que el recorrido del arreglo nos permite aplicar la misma operación a todos los elementos sin necesidad de declarar múltiples variables individuales. Esto demuestra una de las principales ventajas del uso de arreglos: facilitar el procesamiento conjunto de grandes cantidades de datos.

Arreglos en diagrama de flujo

Si quisiéramos representar el uso de arreglos dentro de un diagrama de flujo, el proceso seguiría una secuencia lógica similar a la utilizada en un lenguaje de programación.

En primer lugar, sería necesario declarar o dimensionar el arreglo, indicando la cantidad de posiciones que se van a utilizar. Luego, se establecerían las variables auxiliares necesarias para la carga o procesamiento de los datos.

Posteriormente, se emplearía una estructura repetitiva para recorrer el arreglo. Dentro del ciclo, se realizarían las operaciones correspondientes, como la asignación de valores a cada posición, la lectura de datos o la impresión en pantalla.

Finalmente, una vez que el ciclo completa todas las iteraciones previstas, el algoritmo concluye con el arreglo completamente cargado o procesado.

Por ejemplo, si quisiéramos cargar un arreglo con los primeros cinco números naturales y luego mostrarlos en pantalla, el procedimiento lógico dentro de un diagrama de flujo sería el siguiente.

1. Declaración del arreglo

Se declara un arreglo llamado Arr con capacidad para 5 elementos.



```
DIMENSION Arr[5]
```

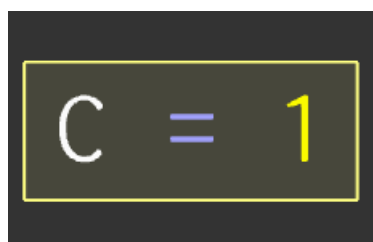
En pseudocódigo se utiliza indexación que comienza en 1, por lo tanto las posiciones válidas del arreglo son:

- Arr[1]
- Arr[2]
- Arr[3]
- Arr[4]
- Arr[5]

2. Inicialización de la variable auxiliar

Se declara e inicializa una variable llamada C con valor 1.

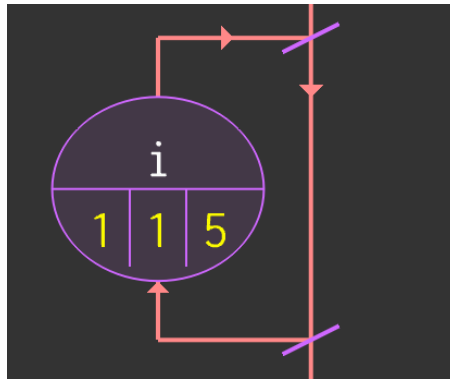
Esta variable se utilizará para asignar valores crecientes a cada posición del arreglo.



```
C = 1
```

3. Inicio del ciclo

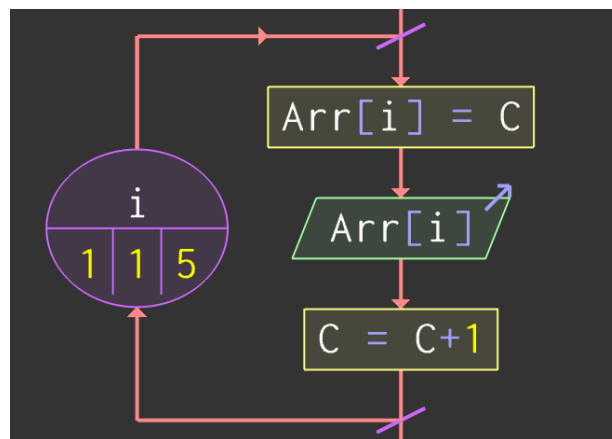
El ciclo comienza con $i = 1$ y finalizará cuando $i = 5$, aumentando en 1 en cada vuelta.



4. Asignación en cada iteración

En cada vuelta del ciclo se ejecuta:

- **Arr[i] = C**
Se guarda el valor actual de C en la posición i del arreglo.
- **Escribir Arr[i]**
Se muestra en pantalla el valor recién almacenado.
- **C = C + 1**
Se incrementa la variable auxiliar para que el siguiente elemento reciba el valor siguiente.



5. Finalización

Una vez que i supera el valor 5, el ciclo termina.

El arreglo queda cargado con los valores:

1, 2, 3, 4, 5

Y esos mismos valores fueron mostrados en pantalla durante el recorrido.

```
C:\Arreglos.exe  - □ X
*** Ejecución Iniciada. ***
1
2
3
4
5
*** Ejecución Finalizada. ***
```

Diagrama de flujo completo:

